FreeRTOS Lab 1

OS-Lab Email : oslab@mail.csie.ncku.edu.tw
TA: p76144906@gs.ncku.edu.tw

Outline

- Requirements
- STM32 Ports
- Tasks
- Queue
- Button debounce
- Grading

Requirements

- Use FreeRTOS multi-task and intertask communications.
- Use button to control LEDs blinking.
  - Implement button debounce function to avoid multiple rapid detections.
- Demo Video

Requirements

There are two tasks: `LED-task` and `Button-task`

`LED-task` have two states (S1, S2):

- **S1**: First, the Red LED lights up for 1 second, followed by the Orange LED lighting up for 1 second, then the Green LED lights up for 1 second. This sequence repeats, cycling through the Red, Orange, and Green LEDs. Only one LED is on at any given time.

- **S2**: Only Orange LED is blinking for 0.5 sec.

Requirements

`Button-task` detected whether the button is pressed.

- If the button is **short-pressed**, the LED-task will change its state (S1→S2) and (S2→S1).
- If the button is **long-pressed** (> 1sec), the LED-task will be suspended, and long pressed again will resume the LED-task.

STM32 Ports

LEDs

- LD1 COM: The LD1 default status is red. LD1 turns to green to indicate that communications are in progress between the PC and the ST-LINK/V2-A.
- LD2 PWR: The red LED indicates that the board is powered.
- User LD3: The orange LED is a user LED connected to the I/O PD13 of the STM32F407VGT6.
- User LD4: The green LED is a user LED connected to the I/O PD12 of the STM32F407VGT6.
- User LD5: The red LED is a user LED connected to the I/O PD14 of the STM32F407VGT6.
- User LD6: The blue LED is a user LED connected to the I/O PD15 of the STM32F407VGT6.
- USB LD7: The green LED indicates when V_{BUS} is present on CN5 and is connected to PA9 of the STM32F407VGT6.
- USB LD8: The red LED indicates an over-current from V_{BUS} of CN5 and is connected to the I/O PD5 of the STM32F407VGT6.

Push-buttons

- B1 USER: The user and wake-up buttons are connected to the I/O PA0 of the STM32F407VG.
- B2 RESET: The push-button connected to NRST is used to RESET the STM32F407VG.

STM32 Ports - Pin Assignments

Configure the LED

- Left-click on PD12, set it to GPIO_Output (it will turn green).
- Right-click on PD12, select Enter User Label, and name it GREEN_LED (or any preferred name).
- Repeat the process for PD14 (RED_LED) and PD13 (ORANGE_LED).

Configure the Button

- Left-click on PA0, set it to GPIO_Input (it will turn green).
- Right-click on PA0, select Enter User Label, and name it BLUE_BUTTON (or any preferred name).

Important Note

- Label names can be customized but avoid spaces
- Press Ctrl + S to save your settings
- Remember to comment out the handlers discussed in Lab 0

STM32 Ports

Save the .ioc file and generate code.

STM32 Ports - APIs

Read and write GPIO by utilizing the following APIs.

```
GPIO_PinState HAL_GPIO_ReadPin(const GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);
void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState);
void HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);
```

Example of lights–up Red LED:

```
HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, GPIO_PIN_SET);
```

Task

- A Task is a small program should never return so are typically implemented as infinite loop.
- A task should have the following structure

```
void vATaskFunction( void *pvParameters ) {
    for( ;; ) {
        // -- Task application code here. --
    }
}
```

Task - Creation

A task is created by `xTaskCreate`

Parameters list:

`pvTaskCode`	Pointer to the task entry function.
`pcName`	A descriptive name for the task.
`uxStackDepth`	The number of words (not bytes!) to allocate for use as the task's stack.
`pvParameters`	A value that is passed as the paramater to the created task.
`uxPriority`	The priority at which the created task will execute.
`pxCreatedTask`	Used to pass a handle to the created task.

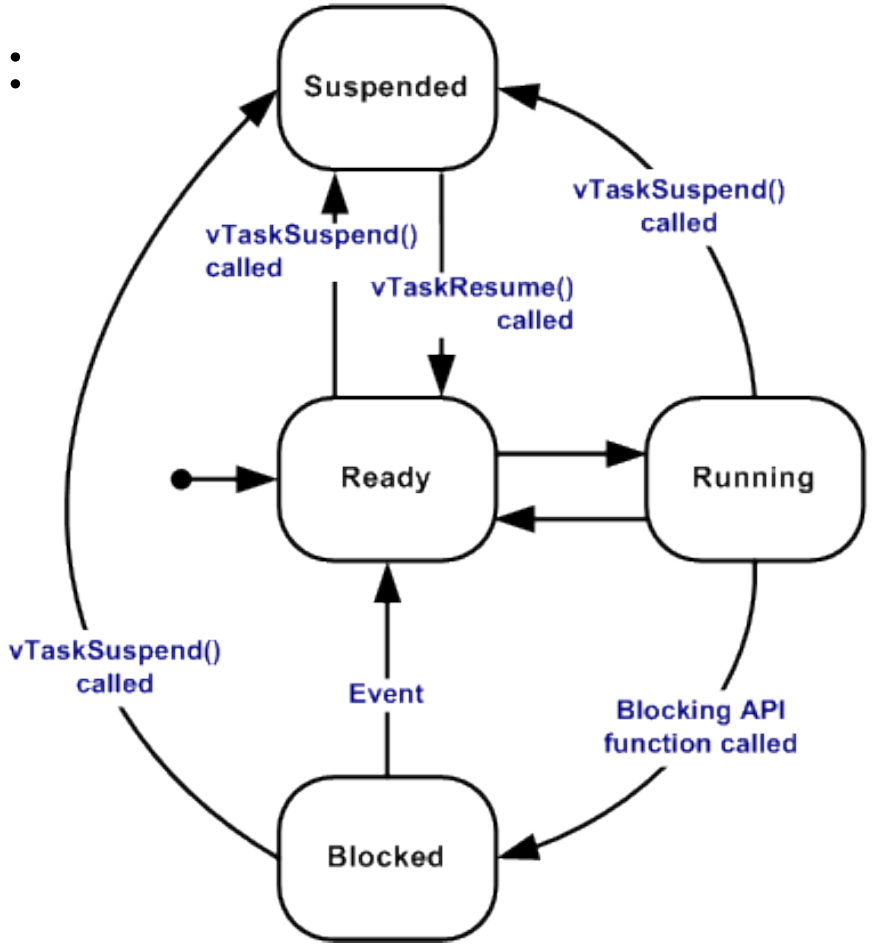
Example of creating a task:

```
TaskHandle_t xLEDHandle = NULL;
xTaskCreate(ledTask, "LEDTask", 128, NULL, 1, xLEDHandle);
```

Task - State

A task can exist in one of the following states:

- Running
- Ready
- Suspended
- Blocked



Example of suspend a task (`vTaskSuspend`):

```
vTaskSuspend(xLEDHandle);
```

Task - Priorities

Each task is assigned a priority from 0 to `(configMAX_PRIORITIES - 1)`, where `configMAX_PRIORITIES` is defined within FreeRTOSConfig.h.

Low priority numbers denote low priority tasks. The idle task has priority zero.

```
#define configTICK_RATE_HZ ((TickType_t
#define configMAX_PRIORITIES (56)
#define configMINIMAL_STACK_SIZE ((uint
```

Task - Scheduler

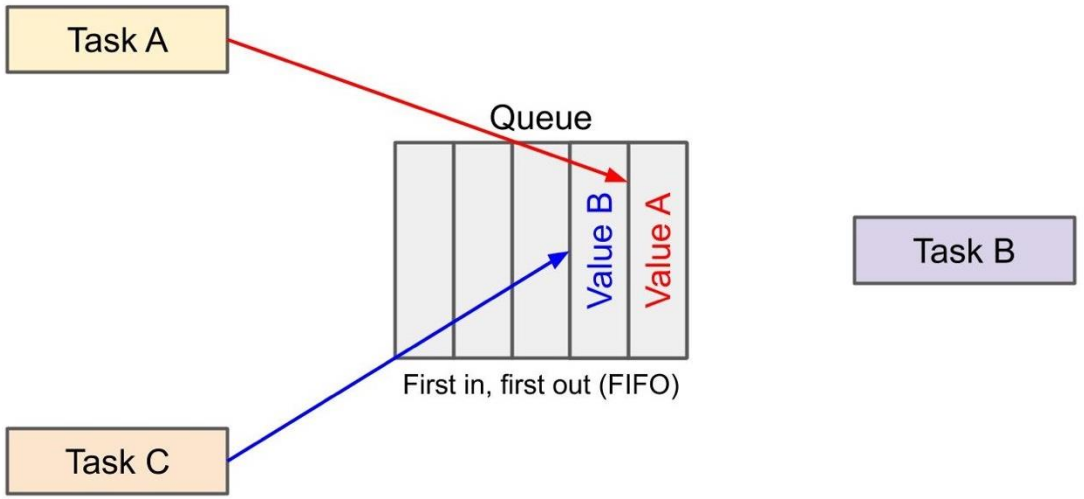
`vTaskStartScheduler` starts the RTOS scheduler. After calling the RTOS kernel has control over which tasks are executed.

```
xTaskCreate(LEDTask, "LED Task", 128, NULL, 3, &LEDHandler);
xTaskCreate(ButtonTask, "Button Task", 128, NULL, 1, &ButtonHandler);
vTaskStartScheduler();

while (1) {
  // Should never reach here
}
```

Queue

- Queues are the primary form of intertask communications.
- Messages are sent through queues by copy not by reference.
- FreeRTOS does not define the message structure, allow users to define.
  - Small messages that are already contained in C variables (integers, small structures, etc.) can be sent into a queue directly.
  - For large data, a message can be just a pointer to the data item to avoid large copying overhead.



Queue

Creates a new queue and returns a handle by which the queue can be referenced.

Example of creating a queue contain 5 elements:

```
xQueueHandle queueHandler;
queueHandler = xQueueCreate(5, sizeof(uint8_t));
```

Queue

xQueueSend and xQueueReceive can be used in send/receive an element.

Sender

```
uint8_t signal = 1;
xQueueSend(queueHandler, &signal, 0);
```

Receiver

```
uint8_t recv;
if (xQueueReceive(queueHandler, &recv, 0) == pdPASS) {
  /* recv now contains a copy of xMessage. */
}
```

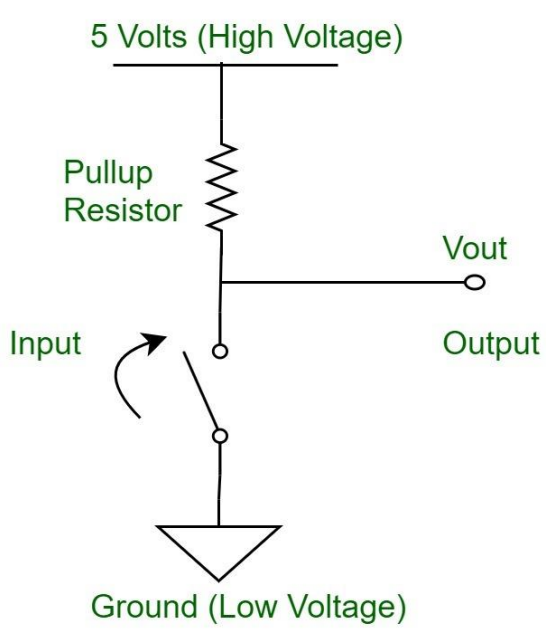
Queue

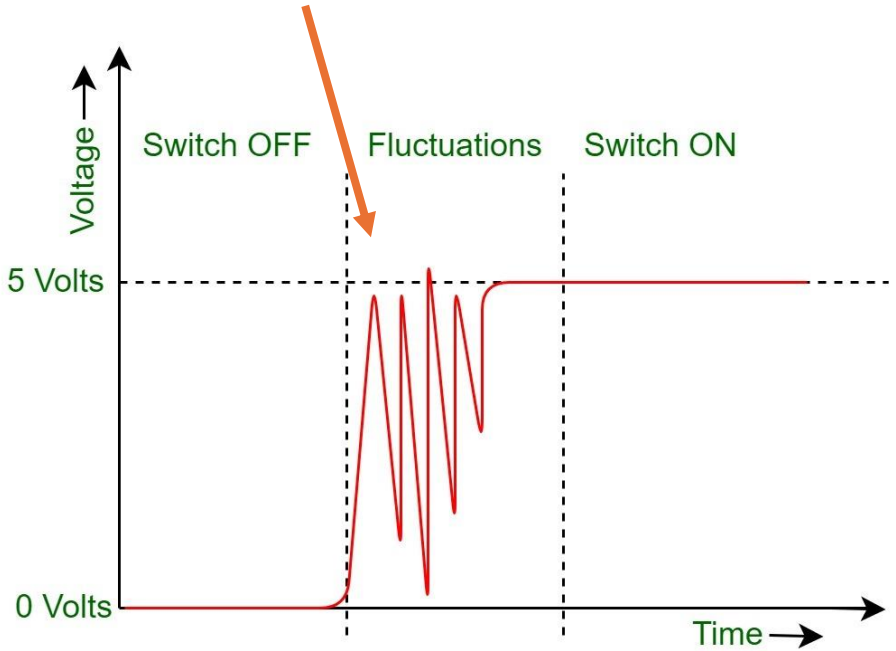
Blocking on queue

- Sending to a full queue → block until space is available or timeout occurs.
- Receiving from an empty queue → block until data arrives or timeout occurs.
- If multiple tasks are blocked on the same queue, the highest-priority task is unblocked first.
- Parameter `xTicksToWait` is the maximum amount of time the task should block wait.

Button Debounce

Mechanical switches bounce when pressed, causing multiple rapid detections instead of a single press. This can lead to unintended multiple task executions





Button Debounce

There are many ways implement software debouncing.

e.g. Adding a delay after detecting a button press.

vTaskDelay: Delays the task in ticks, allowing other tasks to run.

```
while (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET) {
    vTaskDelay(10);
    while (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET &&
```

Grading

Including **Demo** and **Report**

- **Demo** (5%)
  - (1%) At least one LED state is correct.
  - (1%) The LED state changes after pressing the button.
  - (1%) The LED state matches the lab requirements after short-pressed the button.
  - (2%) The LED-task suspend after button been long-pressed.
- **Report** (3%)
  - Write according to the format given on Moodle.
  - Please submit a PDF file with the file name as `studentID_labNo.pdf`
  - e.g. P76144906_lab1.pdf
- The Report must be uploaded on Moodle before 23:59 on 3/27; submissions after the deadline will not be graded.

Useful Links

- UM1472 User manual - Discovery kit with STM32F407VG MCU
- Task Creation – FreeRTOS
- Task Control - FreeRTOS
- Intertask Communications - FreeRTOS